

Check Number Is prime or not in $O(\sqrt{n})$:

```
bool isPrime(int n) {
    if (n <= 1)
        return false;
    if (n <= 3)
        return true;
    if (n % 2 == 0 || n % 3 == 0)
        return false;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    }
    return true;
}
```

Sieve $O(n \log(n) \log(n))$:

```
const int N = 1e6 + 5;
vector<bool> isPrime(N, true);

void sieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= n; ++i) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i)
                isPrime[j] = false;
        }
    }
}
```

Euler Sieve $O(n)$:

```
const int N = 1e6 + 5;
vector<int> primes;
vector<bool> isPrime(N, true);

void eulerSieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i <= n; ++i) {
```

```

        if (isPrime[i]) primes.push_back(i);
        for (int p : primes) {
            if (i * p > n) break;
            isPrime[i * p] = false;
            if (i % p == 0) break;
        }
    }
}

```

Count Divisors of One Number $O(\sqrt{n})$:

```

int countDivisors(int n) {
    int cnt = 0;
    for (int i = 1; i * i <= n; ++i) {
        if (n % i == 0) {
            cnt++; // i
            if (i != n / i) cnt++; // n / i (avoid double count for squares)
        }
    }
    return cnt;
}

```

Count Divisors of All Numbers from 1 to n $O(n \log n)$:

```

const int N = 1e6 + 5;
vector<int> divisors(N, 0);

void countAllDivisors(int n) {
    for (int i = 1; i <= n; ++i)
        for (int j = i; j <= n; j += i)
            divisors[j]++;
}

```

Prime Factorization in $O(\sqrt{n})$:

```

map<int, int> primeFactorize(int n) {
    map<int, int> factors;
    for (int i = 2; i * i <= n; ++i) {
        while (n % i == 0) {
            factors[i]++;

```

```

        n /= i;
    }
}
if (n > 1) factors[n]++;
return factors;
}

```

Prime Factorization in $O(n \log n)$:

```

const int N = 1e6 + 5;
vector<int> spf(N); // smallest prime factor

void sieve() {
    for (int i = 0; i < N; i++) spf[i] = i;

    for (int i = 2; i * i < N; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j < N; j += i)
                if (spf[j] == j) spf[j] = i;
        }
    }
}

vector<int> getFactorization(int x) {
    vector<int> factors;
    while (x != 1) {
        factors.push_back(spf[x]);
        x /= spf[x];
    }
    return factors;
}

const int N = 2e7 + 5;
int spf[N];
vector<int> primes;
vector<bool> isPrime(N, true);

void eulerSieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i <= n; ++i) {
        if (isPrime[i]) primes.push_back(i), spf[i] = i;

        for (int p : primes) {
            if (i * p > n) break;
            spf[i * p] = p;
            isPrime[i * p] = false;
            if (i % p == 0) break;
        }
    }
}

```

```

    }
}

```

Prime Factorization using all priems pre calc factorize 1e9 more fast:

```

vector<int> factorize(int x, const vector<int>& primes) {
    vector<int> res;
    for (int p : primes) {
        if (1LL * p * p > x) break;          // Stop if prime^2 > x
        if (x % p == 0) {
            res.push_back(p);                // p is a prime factor
            while (x % p == 0) x /= p;        // Remove all occurrences of p
        }
    }
    if (x > 1) res.push_back(x);            // x is a large prime factor
    return res;
}

```

Fast Power ($O(\log b)$):

```

long long fastPower(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res *= a;
        a *= a;
        b >>= 1;
    }
    return res;
}

```

Fast Power with Mod ($O(\log b)$):

```

long long fastPowerMod(long long a, long long b, long long mod) {
    long long res = 1;

```

```

    a %= mod;
    while (b > 0) {
        if (b & 1)
            res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

```

GCD , LCM:

```

int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}

```

nPr , nCr:

```

const int M = 3e6+10;
int mod = 1e9+7;
int add ( int a , int b)
{
    return (a + b) % mod;
}
int mul ( int a , int b)
{
    return 1LL * a * b % mod;
}
int fp( int b , int p)
{
    if(!p)
        return 1;
    int temp = fp(b,p/2);
    temp = mul(temp,temp);
    if(p&1)
        temp = mul(temp,b);
    return temp;
}

int fact[M], inv[M];

```

```

void build()
{
    fact[0] = fact[1] = inv[0] = inv[1] = 1;
    for (int i = 2; i < M; i++)
    {
        fact[i] = fact[i - 1] * i % mod;
        inv[i] = inv[mod % i] * (mod - mod / i) % mod;
    }
    for (int i = 2; i < M; i++)
        inv[i] = inv[i] * inv[i - 1] % mod;
}
int nCr( int n , int r)
{
    return mul(fact[n],mul(inv[n-r],inv[r]));
}
int nPr(int n, int r)
{
    int ans = nCr(n, r); ans *= fact[r];
    ans %= mod;
    return ans;
}

```

nCr with rec $O(r)$:

```

bool nCr(int n, int r, int limit) {
    r = min(r, n - r);
    __int128 ans = 1;
    for (int i = 1; i <= r; ++i) {
        ans = ans * (n - i + 1) / i;
        if(ans >= limit) return 1;
    }
    return ans >= limit;
}

```

Compute $\phi(n)$ (number of co primes):

```

int phi(int n) {
    int res = n;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            res -= res / i;
        }
    }
}

```

```

    }
    if (n > 1)
        res -= res / n;
    return res;
}

```

Compute $\phi(n)$ for range:

```

const int N = 1e6 + 5;
int phi[N];

void computeTotients() {
    for (int i = 0; i < N; i++) phi[i] = i;
    for (int i = 2; i < N; i++) {
        if (phi[i] == i) {
            for (int j = i; j < N; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}

```

Count sum of divisors to N :

```

int l = 1;
int res = 0 ;
while (l<=n)
{
    int k = n/l ;
    int r = n/k ;

    int sum = ((l+r)*(r-l+1))/2;
    res+= (sum*k);

    l = r+1 ;
}

```

geometric series :

```

// p^0 + p^1 ... p^a
int geometric(int p, int a) {

```

```

int numerator = (binpow(p, a + 1) - 1 + MOD) % MOD;
int denominator = inv(p - 1);
return numerator * denominator % MOD;
}

```

Generate Cnt , Sum , Prod for primes problem :

```

int cnt = 1 , cntTmp = 1 , sum = 1 , prod = 1 ;
for (int i = 0; i < n; i++)
{
    int a , b; cin>>a>>b;

    int cnt_prev = cntTmp;

    sum*= geometric(a,b);
    sum%= mod;

    prod = fastPowerMod(prod,b+1,mod);
    prod%= mod ;

    int tmp = (b * (b + 1) / 2) % (mod - 1);
    tmp = tmp * cnt_prev % (mod-1);

    prod *= fastPowerMod(a,tmp,mod);
    prod%= mod ;

    cnt*= (b+1);
    cnt %= mod ;

    cntTmp *=(b+1);
    cntTmp %=(mod-1);
}

```

Mibous Algo :

```

const int N = 1e5+5;
vector<int> mu(N), isPrime(N, 1);
void mobius() {
    for (int i = 1; i < N; i++) mu[i] = 1;
    isPrime[0] = isPrime[1] = 0;

    for (int i = 2; i < N; i++) {
        if (isPrime[i]) {
            for (int j = i + i; j < N; j += i)

```



```

        isPrime[j] = 0;

        for (int j = i; j < N; j += i)
            mu[j] *= -1;

        for (long long j = 1LL * i * i; j < N; j += 1LL * i * i)
            mu[j] = 0;
    }
}

```

Pollard Rho Template

```

#include <bits/stdc++.h>
using namespace std;

using ull = unsigned long long;
using u128 = __uint128_t;

// ----- fast mul mod -----
ull mul_mod(ull a, ull b, ull mod) {
    return (u128)a * b % mod;
}

// ----- fast pow mod -----
ull pow_mod(ull a, ull d, ull mod) {
    ull res = 1;
    while (d) {
        if (d & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        d >>= 1;
    }
    return res;
}

// ----- Miller-Rabin primality test -----
bool isPrime(ull n) {
    if (n < 2) return false;

    for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
        if (n % p == 0) return n == p;

    ull d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, s++;

    auto check = [&](ull a) {
        ull x = pow_mod(a, d, n);
        if (x == 1 || x == n - 1) return true;
        for (ull r = 1; r < s; r++) {

```

```

        x = mul_mod(x, x, n);
        if (x == n - 1) return true;
    }
    return false;
};

for (ull a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022})
    if (a % n && !check(a)) return false;

return true;
}

// ----- Pollard Rho -----
ull pollard(ull n) {
    if (n % 2 == 0) return 2;

    ull x = rand() % (n - 2) + 2;
    ull y = x;
    ull c = rand() % (n - 1) + 1;
    ull d = 1;

    auto f = [&](ull v) {
        return (mul_mod(v, v, n) + c) % n;
    };

    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd((ull)llabs((long long)x - (long long)y), n);

        if (d == n) return pollard(n);
    }
    return d;
}

// ----- factorization -----
void factor(ull n, vector<ull> &f) {
    if (n == 1) return;
    if (isPrime(n)) {
        f.push_back(n);
        return;
    }

    ull d = pollard(n);
    factor(d, f);
    factor(n / d, f);
}

```

CoPrimes pairs :

```

const int N = 1e6+5;
vector<int> mu(N), isPrime(N, 1);
void mobius() {
    for (int i = 1; i < N; i++) mu[i] = 1;
    isPrime[0] = isPrime[1] = 0;

    for (int i = 2; i < N; i++) {
        if (isPrime[i]) {
            for (int j = i + i; j < N; j += i)
                isPrime[j] = 0;

            for (int j = i; j < N; j += i)
                mu[j] *= -1;

            for (long long j = 1LL * i * i; j < N; j += 1LL * i * i)
                mu[j] = 0;
        }
    }
}

void solve(){
    int n ; cin>>n;
    int arr[n];
    vi freq(N+5) , freqdiv(N+5);
    lp(i,n)cin>>arr[i] , freq[arr[i]]++;
    for (int i = 1; i < N; i++)
    {
        for(int j = i ; j < N ; j += i){
            freqdiv[i]+=freq[j];
        }
    }
    int ans = 0 ;
    for (int i = 0; i < N; i++)
    {
        ans += (freqdiv[i]*(freqdiv[i] - 1)/2)*mu[i] ;
    }
    cout<<ans ln ;
}

int32_t main()
{
    mobius();
    KERO
    solve();
    return 0;
}

```